National University – Clark
Bachelor of Science in Information Technology
A.Y. 2026-2027
1st Term

Iskolaria: A Containerized Scholarship Application System with Automated Docker and Jenkins CI/CD Pipeline

A **Technical Documentation**
Presented in Partial Fulfillment of the Final Requirements for
Systems Integration and Architecture

Presented by: **Team Ka Ba?**

Cabral, Bea Mei P.
David, Nigel James L.
Serrano, Bianca Gabrielle R.
Soriano, Den Mark C.

ITE302

Submitted to:

Mr. Harwin C. Mendoza

October 2026

Table of Contents

I. TEAM COMPOSITION AND PROJECT DESCRIPTION 3
Group Name, Members, and Roles 3
Project Description 3
System Modules and Their Responsibilities 5
II. SYSTEM ARCHITECTURE 9
System Architecture Diagram 9
Architecture Explanation 10
Ports, Networks, and Volumes 10
Request Flow: Submitting an Application 11
III. TECHNOLOGY STACK WITH JUSTIFICATION 12
IV. DOCKERFILE EXPLANATIONS 14
Dockerfile Configurations by Service 14
Security and Design Choices 16
V. DOCKER COMPOSE 17
Services 17
Architectural Highlights 17
VI. JENKINS SETUP 19
Installation Steps, Plugins, and Credentials 19
Trigger Configuration 20
VII. PIPELINE EXPLANATION 20
Failure Handling 22
Post Section 23
VIII. TESTING STRATEGY 23
How failures block deployment 24
Scope 24
IX. ROLLBACK PROCEDURE AND DATA PERSISTENCE 24
Rollback 24
Data Persistence 26
X. PROBLEMS ENCOUNTERED 28
XI. INDIVIDUAL CONTRIBUTIONS 29
Members' Roles and Responsibilities 30
Individual Contributions and Deliverables 30
XII. REFERENCES 31

I. TEAM COMPOSITION AND PROJECT DESCRIPTION

Group Name, Members, and Roles

TEAM KA BA?

Iskolaria: A Containerized Scholarship Application System with Automated Docker and Jenkins CI/CD Pipeline

MEMBER	ROLE	RESPONSIBILITY
Cabral, Bea Mei P.	Infrastructure Engineer	Architecture, Dockerfiles, docker-compose.yml, proxy
David, Nigel James L.	DevOps / CI-CD Engineer	Jenkins in Docker, credentials, triggers, Jenkinsfile
Serrano, Bianca Gabrielle R.	Project Lead / Scrum Master	Proposal, task board, Git repository management, demo flow
	Backend and Database Engineer	API modules, DB schema and seed data, unit tests
Soriano, Den Mark C.	Frontend, QA, and Documentation Lead	Frontend, smoke test, README, documentation, slides

Project Description

Iskolaria is a web-based scholarship management system. Students can browse the scholarships that are open, check if they are eligible, upload the required documents, and submit an application, all in one place. Scholarship officers use a separate review board to create and close scholarships, go through the applications, verify or flag the uploaded documents, and set the status of each application to under review, waitlisted, approved, rejected, or removed from the scholarship. An application that is still pending can also be cancelled by the student who submitted it. Each time

the status of an application changes, a notification is shown to the student inside the system.

The system is divided into several modules, and each one runs in its own Docker container. The frontend provides the pages used by both students and administrators. Registration and login are handled by the auth-service. During registration, the student number is compared with the school's records, and a JWT is issued after a successful login so that the other services can identify the user and the role. The scholarship programs, including their requirements, deadlines, and available slots, are managed by the scholarship-api, which also tells whether a student is eligible. Applications are submitted and tracked through the application-api, which also moves them through the review workflow and provides the notification history and the summary report. Required documents are uploaded through the upload-service, where each file is checked for type and size and where administrators can open and verify the files. The worker runs in the background. Status-change jobs are read from a Redis queue by the worker, and the matching notifications are saved to the database. Every request enters through an Nginx reverse proxy, which is the only public entry point of the system. The persistent data is kept by PostgreSQL in a named volume, so the records are not lost when containers are restarted or redeployed.

The project is built, tested, and deployed automatically with Docker, Docker Compose, and Jenkins. Jenkins runs in its own container, apart from the application stack, and the pipeline is written in a Jenkinsfile that is stored in the repository. A push to GitHub starts the pipeline. In the current setup, the repository is checked every minute by Poll SCM, and a build is started when a new commit is found. A GitHub webhook trigger is also declared in the Jenkinsfile. The stages are run in this order:

1. Checkout: The pushed code is retrieved from the repository.
2. Test: The automated tests are run inside Docker. If a test fails, the pipeline stops, so broken code never reaches deployment.
3. Build Images: The Docker images are built and tagged with the build number.
4. Deploy: The updated stack is started with Docker Compose, using the secrets kept in Jenkins Credentials.
5. Smoke Test: The health of the services is checked through the reverse proxy.

The result of every build is sent to Discord. Since each build keeps its own tagged images, an earlier version can be redeployed at any time, and no data is lost when this is done.

System Modules and Their Responsibilities

Module (Container)	Technology	Responsibility and Main Endpoints
proxy	**Nginx 1.27**	The proxy is the single public entry point of the system. It is published on port 8080, which is mapped to port 80 inside the container. Requests for the pages are passed to the frontend, while requests that begin with /api/auth/, /api/scholarships/, /api/applications/, or /api/uploads/ are passed to the matching service.
frontend	**Static HTML/CSS/ JS on Nginx**	The frontend is made up of six static pages: the scholarship list (index), registration (register), login (login), the application form (apply), the student's own applications and notifications (my), and the review board for administrators (admin). The shared code for the session, the API calls, and the navigation bar is kept in api.js, while the build number shown in the footer is set by build.js.
auth-service	**Python 3.12; FastAPI**	Registration is handled by POST /register, where the student number is compared with the students table before an account is created. Login is handled by POST /login, which returns a JWT together with the role and the full name of the user. GET /me returns the id and the role found in the token, and GET /health lets Docker know that the container is alive. Passwords are hashed with bcrypt, and the tokens expire after one hour. The administrator account is created automatically at startup from the values in the environment file.

scholarship-api	**Python 3.12; FastAPI**	A new scholarship is created through POST /scholarships, and its remaining slots start out equal to the total slots. POST /eligibility/check compares the GWA, family income, and year level of a student with the requirements of the scholarship, together with its deadline, status, and slots, and returns every reason why a check failed. GET /scholarships lists only the open scholarships, with the nearest deadline first, while GET /scholarships/all lists both open and closed ones and is meant for administrators. GET /scholarships/{id} returns the details of one scholarship or a 404 if it does not exist, and GET /health is used as the liveness check. A scholarship is edited through PUT /scholarships/{id}. When the total number of slots is changed, the remaining slots are shifted by the same difference and never go below zero.
application-api	**Node.js 20, Express**	An application is submitted through POST /applications. The eligibility and the required documents are checked first, then the application is saved and a reference number such as SCH-2026-00001 is returned. A second application to the same scholarship is rejected unless the earlier one was cancelled. GET /applications/mine lists the applications of the student with their statuses and remarks, and GET /notifications/mine lists the notifications of the student, newest first. For administrators, GET /applications lists all applications and can be filtered by status or scholarship, and GET /reports/summary gives the totals per status, the slots of each scholarship, and the number of applications received today. GET /health reports whether the service is running. A student can cancel an application through PUT /applications/{id}/cancel while it is submitted, under review, or waitlisted. Administrators move an application through the allowed review flow with PUT

		/applications/{id}/status. An approval takes one slot, the removal of an approved student gives the slot back, and every change places a notification job in the queue. Both actions are refused once the scholarship has been closed.
upload-serviceservice	**Python 3.12; FastAPI**	Files are sent through POST /upload, which accepts PDF, JPG, or PNG files of up to 2 MB. The document type must be one that the scholarship requires, and the real file format is checked from the first bytes of the file and not from its name. Each file is saved under a random name, and uploading the same document type again replaces the earlier file. GET /documents/mine lists the documents of the student for a scholarship together with their review status. GET /documents lets an administrator list the documents of an applicant, and GET /files/{id} downloads a file, which only its owner or an administrator may do. A document is marked as verified or flagged by an administrator through PUT /documents/{id}/review, and GET /health is used for the health check. The files are stored in the uploads-data volume, while the document records are kept in PostgreSQL.
worker	**Python 3.12**	The worker has no public endpoints. Jobs are taken from the notifications list in Redis, moved to a processing list while they are being handled, and the matching message is saved to the notifications table in PostgreSQL. A job that is invalid or cannot be saved because of bad data is moved to a dead-letter list, and when the database cannot be reached, the save is retried every three seconds. Jobs left unfinished by a crash are returned to the queue when the worker starts again. A small HTTP server that answers GET /health on port 8000 is run only for the Docker health check. It reports a failure when the worker has not completed a cycle in the last 30 seconds.

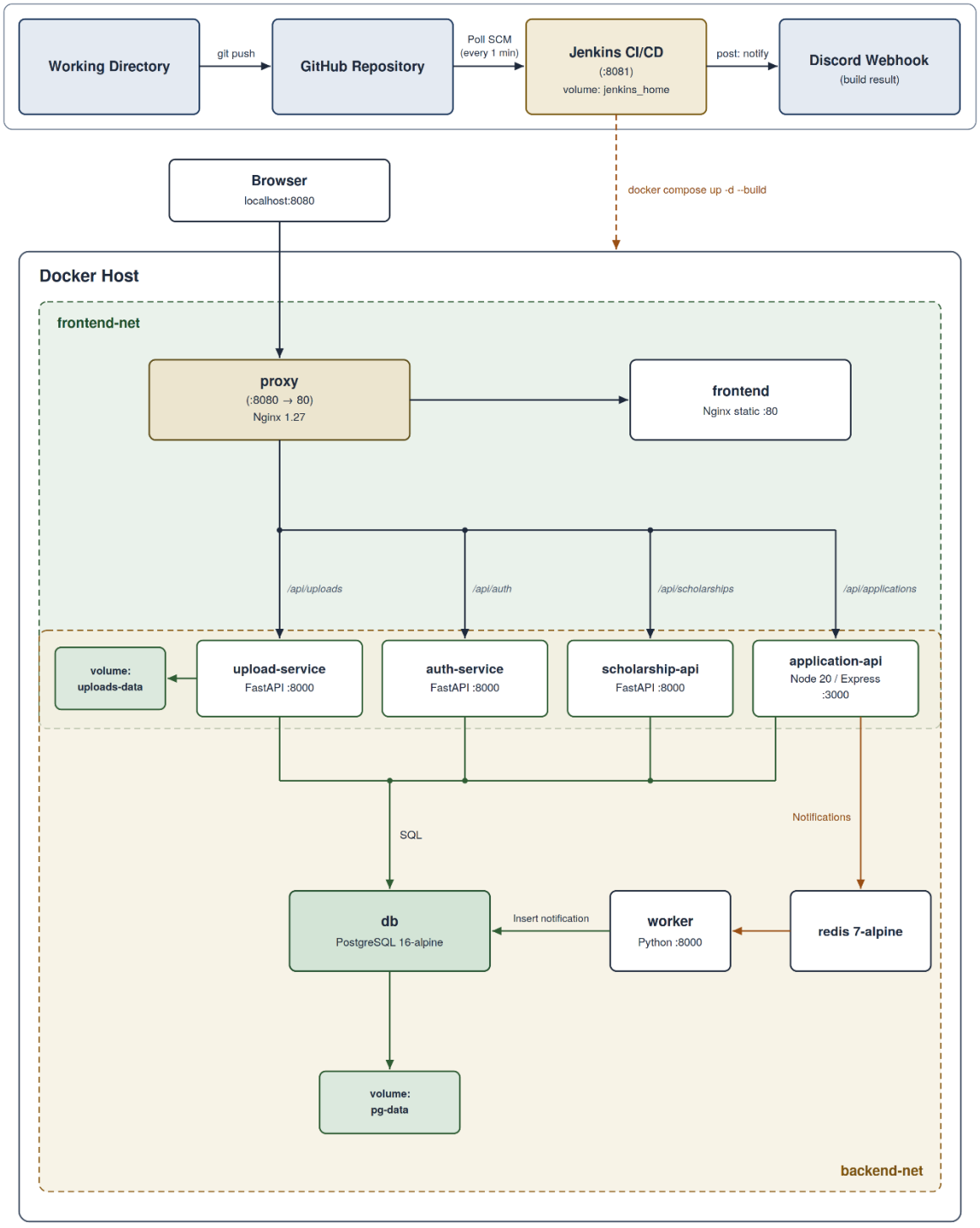
db	**PostgreSQL 16 (Alpine)**	The database stores the data of the system and can only be reached by the backend services. The tables are students, users, scholarships, applications, documents, and notifications. Constraints prevent invalid data, such as a second application from the same student to the same scholarship. The tables and the starting data are created by init.sql the first time the container starts. The data is kept in the pg-data volume, so it remains after a redeployment. PostgreSQL listens on port 5432 inside backend-net and cannot be reached from outside.
redis	**Redis 7 (Alpine)**	Redis serves as the queue between the application-api and the worker. Three lists are used: notifications for the jobs that are waiting, notifications:processing for the jobs that are being handled, and notifications:dead for the jobs that failed. Because of this queue, a response can be returned to the user right away while the notifications are processed in the background. Redis listens on port 6379 inside backend-net, is not exposed to the host, and has no endpoints of its own.
Jenkins (separate compose)	**Jenkins LTS (JDK 17) + Docker CLI**	Jenkins runs the CI/CD pipeline from its own compose project, so redeploying the application never restarts Jenkins. The pipeline goes through Checkout, Test, Build Images, Deploy, and Smoke Test, and a failed test stops the deployment. The images are tagged with the build number so that an earlier version can be redeployed, the deployment secrets are stored in Jenkins Credentials, and the result of every build is sent to Discord. The web interface is available on port 8081, the agents use port 50000, and the data is kept in the jenkins_home volume. The Docker engine of the host is reached through the mounted docker.sock.

II. SYSTEM ARCHITECTURE

System Architecture Diagram

Figure 1

Iskolaria System Architecture



Architecture Explanation

Iskolaria is a scholarship application system split into small containers that run together with Docker Compose. When a developer pushes code to GitHub, Jenkins detects the change, rebuilds the containers and sends the build result to Discord. Students use the system through one address, localhost:8080. The proxy (Nginx) receives every request. It sends page requests to the frontend and sends API requests to the right service: auth-service for login, scholarship-api for scholarships, application-api for applications, and upload-service for file uploads. The services save their data in one PostgreSQL database.

When an application is submitted, application-api puts a notification job in Redis. The worker takes the job and saves the notification in the database, so the student does not have to wait for it.The proxy and frontend are kept on a separate network from the database, Redis and worker, so only the services can reach the data.

Ports, Networks, and Volumes

ITEM	DETAIL
Published ports	Only port 8080 is published to the host, and it is mapped to port 80 of the proxy. All other application traffic stays inside Docker. Jenkins is deployed separately, and its web interface (8081) and agent port (50000) are published by its own compose file.
Internal ports	The other services listen only on the internal Docker networks and cannot be reached from the host. The frontend is served on port 80, while the auth-service, scholarship-api, upload-service, and the health endpoint of the worker use port 8000. The application-api uses port 3000, the database uses port 5432, and Redis uses port 6379.
frontend-net	This network carries the traffic that comes from outside. The proxy, the frontend, and the four APIs (auth-service, scholarship-api, application-api, and upload-service) are attached to it, so that the requests received by the proxy can be forwarded to the correct service. The database and Redis are not on this network, so the proxy cannot reach them directly.
backend-net	This network carries the traffic of the data tier. The database, Redis, and the worker are attached only to this network, while the four APIs are attached to both networks. The APIs are therefore the only link between the proxy and the stored data.

Volumes	Three named volumes are used so that the data is kept when containers are replaced. The volume pg-data holds the PostgreSQL data, uploads-data holds the uploaded documents (mounted at /data/uploads in the upload-service), and jenkins_home holds the Jenkins configuration and build history.
Project name	The Compose project is named iskolaria. The name is set in docker-compose.yml (name:) and in the Jenkinsfile (COMPOSE_PROJECT_NAME). Because of this, every redeployment reuses the existing containers and volumes instead of creating new, empty ones.

Request Flow: Submitting an Application

The browser sends its requests to the proxy at http://localhost:8080. The static pages are served by the frontend, and every request that begins with /api/ is passed to the matching service after the prefix has been removed by the proxy.

- On the apply page, each required document is first uploaded to the upload-service. A file cannot be larger than 2 MB, and its type (PDF, JPG, or PNG) is verified from its signature bytes.
- The application is then submitted to POST /applications of the application-api. The scholarship-api (/eligibility/check) is asked to check the scholarship status, deadline, available slots, GWA, family income, and year level. After the presence of all required documents is confirmed, the application is saved. Duplicate applications are prevented by a UNIQUE constraint on the user and the scholarship.
- A JSON job is pushed by the application-api to the Redis list “notifications”, and a reference number (for example, SCH-2026-00001) is returned to the student.
- The worker moves the job atomically to a processing list, writes the notification row to PostgreSQL, and then removes the job. The resulting message is shown to the student on my.html.
- When the status is changed by an administrator, the same queue is used. On approval, slots_remaining is decreased inside the same database transaction through an atomic UPDATE that requires slots_remaining > 0.

III. TECHNOLOGY STACK WITH JUSTIFICATION

Iskolaria is built as a set of small services written in different languages. The purpose is to show that containers allow different runtimes to live together and communicate with each other without trouble. Python (FastAPI) and Node.js (Express) services each run in their own container and exchange plain HTTP/JSON, with a signed JWT carrying the user's identity and role, so no service needs to know what language another one uses. Nginx, PostgreSQL and Redis support them, and Docker Compose with a Jenkins pipeline starts, tests and deploys the whole stack. The table below lists each component and why it was chosen.

Component	Choice	Justification
Application Idea	Scholarship application system	The idea can be divided easily into separate modules: identity management, the scholarship catalog, application transactions, file handling, and asynchronous notifications. This satisfies the requirement of having more than one module.
Languages & Frameworks	Node.js 20 (Express) and Python 3.12 (FastAPI)	Using two languages shows that containers allow different runtimes to run side by side. FastAPI gives typed request validation with very little code, while Express is well suited for handling asynchronous HTTP calls and Redis operations.
Frontend	Static HTML/CSS/JS	No build step is needed, so the image stays small and starts quickly. The Dockerfile writes build.js during the image build using the Jenkins build number, so the footer shows which version is running.
Reverse Proxy	Nginx 1.27 Alpine	It is lightweight, gives a single public entry point (port 8080), and routes requests by path in a simple way. The configuration is copied directly into the image.

Database	PostgreSQL 16 Alpine	It was selected for its relational integrity, its support for UNIQUE constraints, and its safe handling of atomic transactions. The data is kept in the named Docker volume pg-data.
Queue	Redis 7 Alpine	The notification process is separated from the request path of the user. Redis lists with BLMOVE give a reliable queue, and failed jobs can be moved to a dead-letter list.
Authentication	JWT (HS256) and bcrypt	The services can verify the identity and the role of a user with a shared secret, without querying the database on every request. Bcrypt is used so that passwords are stored only as secure hashes.
Containers & Orchestration	Docker and Docker Compose	The application can be started with one command. Each service defines a health check, and Docker Compose uses depends_on with condition: service_healthy so that services start in the correct order.
CI/CD Pipeline	Jenkins LTS in Docker	A declarative Jenkinsfile keeps the pipeline configuration under version control together with the application code.
Git Hosting & Trigger	GitHub, webhook, and Poll SCM	Poll SCM checks the repository every minute and starts a build when a new commit is found, so the pipeline does not depend on the webhook. A GitHub webhook trigger is also declared in the Jenkinsfile for the case where Jenkins can be reached from GitHub.

Testing	Jest (application-api) and Pytest (Python services)	Each language uses its own native test tool, and the tests are run automatically inside a dedicated Docker build target. Because of this, no language dependencies have to be installed on the Jenkins host.
Notifications	Discord webhook	The build results are sent to the development team right away. The webhook URL is kept in Jenkins Credentials, so it is never written in the code.

IV. DOCKERFILE EXPLANATIONS

Every custom module has its own Dockerfile and .dockerignore file, which help keep the images small and the builds fast. The five backend services follow the same multi-stage build pattern, consisting of a base stage where dependencies are installed, a test stage where unit tests are executed, and a production stage that becomes the final image.

During the Test stage of the CI/CD pipeline, Jenkins builds only the test target using docker build --target test. This allows the automated tests to run inside Docker without requiring the application dependencies to be installed directly on the Jenkins host.

The base images are selected according to the requirements of each service. Node.js and Python images support the backend applications, Nginx serves the frontend and reverse proxy, PostgreSQL provides database storage, and Jenkins uses a custom image that includes the tools needed for the CI/CD pipeline.

Dockerfile Configurations by Service

Each service has a Dockerfile configured according to its function and runtime requirements.

Application API

The application-api uses node:20-alpine as its base image. Its production stage runs as the unprivileged node user, reducing the risks associated with running the

application as root. The command npm ci --omit=dev installs the production dependencies while excluding development dependencies. A HEALTHCHECK is also included, using wget to request localhost:3000/health.

Authentication Service, Scholarship API, and Worker

The auth-service, scholarship-api, and worker use python:3.12-slim as their base image. A dedicated appuser is created so that the applications do not run as root. Dependencies are installed using --no-cache-dir to reduce unnecessary package cache files in the image. Their health checks use Python's urllib to request localhost:8000/health.

Upload Service

The upload-service also uses python:3.12-slim. Its Dockerfile creates the /data/uploads directory and assigns ownership to the non-root appuser. This allows the service to write uploaded documents to the directory mounted by the uploads-data named volume.

Frontend and Reverse Proxy

Both the frontend and proxy use nginx:1.27-alpine and follow a single-stage build approach. For the proxy, nginx.conf is copied directly into the image to provide the routing configuration and avoid deployment issues associated with bind mounts when Jenkins manages the application stack. For the frontend, the Dockerfile uses the ARG BUILD_NUMBER argument to generate build.js. This allows the Jenkins build number to appear in the website footer, helping identify the version currently deployed.

Database

The database uses postgres:16-alpine. The init.sql file is copied into /docker-entrypoint-initdb.d/, allowing PostgreSQL to initialize the database schema and starting data when the database volume is first initialized. These initialization scripts are not automatically rerun every time an existing database container restarts.

Jenkins

The Jenkins service uses a custom Dockerfile located in infra/jenkins, extending jenkins/jenkins:lts-jdk17. The Dockerfile installs the Docker CLI and Docker Compose plugin using apt-get, enabling Jenkins to build images and manage the application containers.

The Jenkins container is configured to run as root in its Compose configuration so that it can access the host's Docker engine through the mounted /var/run/docker.sock. This configuration is intended for the controlled laboratory environment because root access and Docker socket access provide extensive control over the host.

Security and Design Choices

Several design choices are applied to improve build efficiency, image consistency, and container security.

Layer Caching

Package files such as package.json and requirements.txt are copied into the image before the application source code. This allows Docker to reuse cached dependency-installation layers when the source code changes but the dependency files remain unchanged, reducing build time.

Reproducible Versions

Base images use specified version tags, such as node:20-alpine and python:3.12-slim. Python packages are also pinned to specific versions, such as fastapi==0.115.6. These choices help reduce unexpected compatibility issues caused by dependency updates. Version tags alone do not guarantee completely reproducible builds because the underlying images may change; image digests can provide stricter version control.

Non-Root Execution

The backend services are configured to run as non-root users where applicable. The application-api uses the unprivileged node user, while the Python services use a dedicated appuser. This follows the principle of least privilege and reduces the potential impact of a compromised application.

Configuration Baked into Images

Files such as nginx.conf and init.sql are copied directly into their respective images. This ensures that the required configuration files are available when the containers start and reduces dependence on external file mounts for these configurations.

V. DOCKER COMPOSE

The docker-compose.yml file, whose project name is iskolaria (set through the name: attribute), orchestrates nine services, two user-defined networks, and two named volumes. Once the .env file is in place, the whole system can be started in the correct order with docker compose up -d --build. In the pipeline, Jenkins builds the images first and then deploys them with docker compose up -d --no-build.

Services

Service Category	Service Name	Base Image / Target	Internal Port	Primary Dependencies
Edge Routing	proxy	nginx:1.27-alpine	80 (Exposed: 8080)	frontend, All APIs
User Interface	frontend	nginx:1.27-alpine	80	None
Auth & Identity	auth-service	python:3.12-slim	8000	db
Catalog	scholarship-api	python:3.12-slim	8000	db
Application Core	application-api	node:20-alpine	3000	db, redis, scholarship-api
Storage & Uploads	upload-service	python:3.12-slim	8000	db
Asynchronous Processing	worker	python:3.12-slim	8000 (health endpoint only)	db, redis
Data Tier	db	postgres:16-alpine	5432	None
Messaging Tier	redis	redis:7-alpine	6379	None

Architectural Highlights

Image Tags

Each built service is named iskolaria/<service>:${TAG:-latest}. The TAG variable is supplied by Jenkins as the build number, so every pipeline run keeps its

own versioned images and a rollback is easy. The Python and Node services are built with target: production, so the test stage is never part of the deployed image.

Restart Policy

Every service uses restart: unless-stopped, so containers recover automatically after a crash or host reboot unless they were stopped manually.

Health Checks

The custom services define a HEALTHCHECK in their Dockerfiles. The db service uses pg_isready against 127.0.0.1, so the check passes only when the real PostgreSQL server is ready and not the temporary server used during initialization. Redis uses redis-cli ping.

Dependencies (depends_on).

All dependencies use the service_healthy condition, so a service starts only when what it needs is actually ready.

- The proxy waits for the frontend and the four APIs.
- The application-api waits for db, redis, and scholarship-api.
- The auth-service, scholarship-api, upload-service, and worker wait for db, and the worker also waits for redis.

Networks and Isolation

- frontend-net: contains the proxy, the frontend, and the four APIs.
- backend-net: contains the four APIs, the worker, the db, and redis.
- Security isolation: the proxy is attached only to frontend-net, so traffic from outside cannot reach the database or the Redis queue directly.

Volumes

- pg-data: mounted to /var/lib/postgresql/data to keep the database.
- uploads-data: mounted to /data/uploads in the upload-service.

Both are named volumes, so the data remains after containers are recreated and after pipeline deployments.

Environment Variables and Secrets

The database settings (DB_HOST, DB_NAME, DB_USER, and DB_PASSWORD) are shared by the backend services through a YAML anchor (x-db-env: &db-env). JWT_SECRET, ADMIN_EMAIL, and ADMIN_PASSWORD are passed

only to the services that need them. A .env.example file with placeholder values is committed to Git. During deployment, Jenkins copies a Secret file credential into a real .env file, and this file is deleted in the post section at the end of every run.

Published Ports

Only 8080:80 on the proxy service is published to the host. All other traffic remains internal.

VI. JENKINS SETUP

The Jenkins setup has its own container and its own compose project, found in the infra/jenkins directory. Because of this, the pipeline never redeploys Jenkins, and the Jenkins data volume stays safe when the application stack is rebuilt or taken down.

Installation Steps, Plugins, and Credentials

- Go to the infra/jenkins directory and run docker compose up -d --build. The custom Jenkins image is built (the Docker CLI and the Compose plugin are added on top of jenkins/jenkins:lts-jdk17), and the container is started in the background.
- Port 8081 is published for the Jenkins web interface and port 50000 for the build agents. The named volume jenkins_home keeps the Jenkins data across restarts, and /var/run/docker.sock is mounted so that Jenkins can use the Docker engine of the host. The container runs as root in its Compose file so it can use the Docker socket; this is intended for the lab environment only.
- Retrieve the initial administrator password by running docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword.
- Open http://localhost:8081 in a browser and enter the password on the "Unlock Jenkins" screen.
- Select **Install suggested plugins** and wait until the installation is finished. The suggested set already contains the Git, GitHub, and Pipeline plugins needed by the Jenkinsfile, together with the Credentials Binding plugin used by withCredentials.
- Create the first administrator account when asked, then confirm the Jenkins URL.

- Check that Jenkins can reach the Docker engine of the host by running docker exec jenkins docker ps. The running containers should be listed.
- Go to **Manage Jenkins > Credentials > System > Global credentials > Add Credentials** and create the two entries below. The IDs must be exactly as shown, because the Jenkinsfile refers to them by name.
- Create the pipeline job by selecting **New Item**, entering a job name, and choosing **Pipeline**.
- In the job configuration, set **Definition** to *Pipeline script from SCM* and **SCM** to *Git*. Enter the repository URL, set the branch to main, and set **Script Path** to Jenkinsfile.
- Save the job.

CREDENTIAL ID	TYPE	USED FOR
iskolaria-env	Secret file	The production .env file. It is copied into the workspace in the Deploy stage and removed in the post section at the end of the run.
discord-webhook	Secret text	The Discord webhook URL, used to report whether a build succeeded or failed.

Trigger Configuration

The pipeline starts automatically whenever new code is pushed to the GitHub repository. This is done with the Poll SCM trigger, which checks the repository every minute. When a new commit is found, Jenkins starts a build without any manual action, and the code is tested, built, and deployed. A GitHub webhook trigger (githubPush) is also declared in the Jenkinsfile, but the automatic deployment does not depend on it. The triggers declared in a Jenkinsfile are registered only after the pipeline has been run once, so the first build was started manually. Every later build was started by a push.

VII. PIPELINE EXPLANATION

The pipeline is kept in the same repository as the application code. Five stages are run in sequence: Checkout, Test, Build Images, Deploy, and Smoke Test. Concurrent builds are not allowed, the last 15 builds are kept, and every run is limited

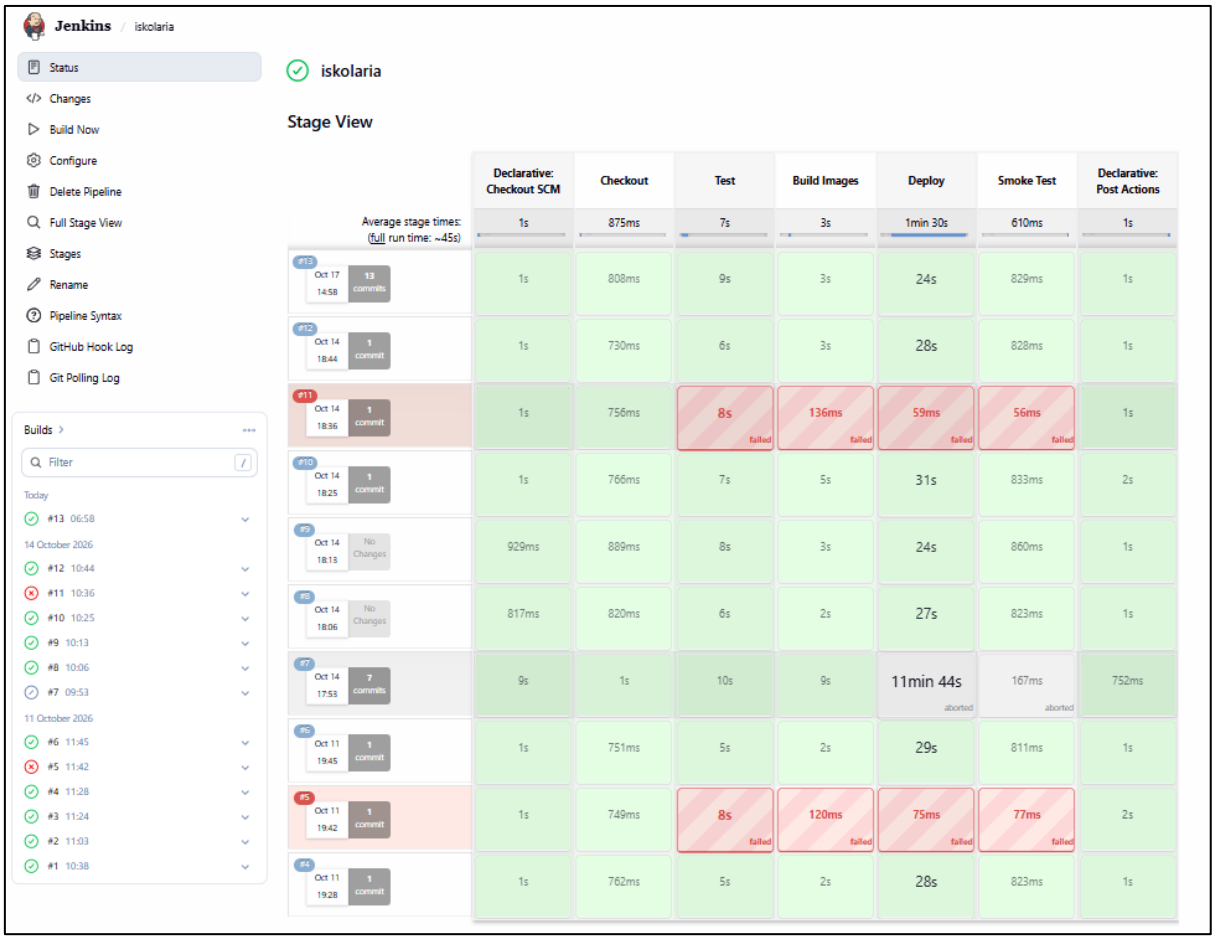
to 30 minutes. Two environment variables are defined for the whole pipeline: COMPOSE_PROJECT_NAME, which is fixed to iskolaria, and TAG, which is set to the Jenkins build number so that each set of images can be identified and redeployed later.

A run starts automatically when new code is pushed to the repository. Both a GitHub webhook trigger (githubPush) and a Poll SCM trigger are declared. With Poll SCM, the repository is checked every minute, so a new commit is still detected even when the webhook cannot reach Jenkins. A manual start is needed only once, so that Jenkins can register the declared triggers.

STAGE	RESPONSIBILITIES
Checkout	The source code of the commit that started the build is retrieved from the repository, and all later stages work on this exact version of the code.
Test	The automated tests of the five backend modules (auth-service, scholarship-api, application-api, upload-service, and worker) are run, each one inside its own docker build --target test.
Build Images	All service images are built with docker compose build and tagged with the build number.
Deploy	The production .env file is copied into the workspace from the Jenkins Secret file credential iskolaria-env, and the stack is started with docker compose up -d --no-build --remove-orphans. Only the containers whose images have changed are replaced. The named volumes are not touched, so the database records and the uploaded documents are kept.
Smoke Test	The deployed system is verified through the reverse proxy. The health endpoints of the authentication, scholarship, application, and upload services are requested together with the home page. The check is repeated every 5 seconds for up to 60 seconds before it is marked as failed.

Figure 2

Jenkins Stage View of the Iskolaria pipeline



Failure Handling

When a test fails, the pipeline stops immediately, and the Build Images and Deploy stages are not run. Defective code is therefore not released, and the running application stays on the previous version. A failure in the Checkout or Build Images stage has the same effect because both come before Deploy.

A failure in the Deploy or Smoke Test stage is different, since these stages run after the new containers have started to replace the old ones. The failure is reported, but the new version may already be running, so recovery is done by redeploying the images of the last working build with TAG=<previous build number> docker compose -p iskolaria up -d --no-build. This needs the real .env file in the project folder, because the pipeline deletes its own copy after every run.

Post Section

It is executed after every run. When the build succeeds, a message saying that the build is live is written to the console, and a notification is sent to a Discord channel. When the build fails, the rollback command is written to the console, and a failure notification is sent. In both cases, the .env file is removed from the workspace and the untagged leftover images are pruned, while the tagged images are kept for rollback.

VIII. TESTING STRATEGY

The automated tests are unit tests of the business rules of each backend service. These rules are kept in separate logic modules that do not need a database, Redis, or the network, so the tests are fast and give the same result on a developer machine and in Jenkins. The Python services use Pytest and application-api uses Jest, for 40 tests in total.

SERVICE	FRAMEWORK	SCOPE OF VERIFICATION
application-api	Jest	The status workflow is tested for allowed and prohibited transitions (review cannot be skipped, an approved application can only be removed, and a removed application is final). The cancellation rules for pending applications and the detection of missing required documents are also tested.
auth-service	Pytest	Identity and credential handling is tested: student number normalization, bcrypt password hashing and verification, and JWT creation and decoding. Tokens that are expired or signed with a different secret must be rejected.
scholarship-api	Pytest	The eligibility evaluation is tested for the GWA limit, family income ceiling, year level, application deadline, and slot availability. Input validation is also tested, where a GWA outside the 1.00 to 5.00 range and a negative income are rejected.
upload-service	Pytest	Upload validation is tested. The file type is determined from the signature bytes (PDF, PNG, and JPEG), executables and renamed text files are rejected, the 2 MB limit is verified, and the content types are matched to the stored extensions.

worker	Pytest	Notification processing is tested. A message exists for every status, a fallback message is used for unknown statuses, and the job validation rejects invalid JSON, missing fields, unknown statuses, and non-integer identifiers.

How failures block deployment

The tests run inside docker build --target test. When a test fails, the build exits with an error, the Test stage turns red, and Jenkins aborts the pipeline, so Build Images, Deploy, and Smoke Test are never run. As a demonstration, a commit that changes the transition table so that submitted can go straight to approved breaks the test "cannot skip review". The build then fails at Test, and the site keeps showing the previous build number. Once the commit is reverted, the next build passes.

Scope

The tests cover the rules that decide what the system accepts or refuses. They do not include integration tests against a real database, API endpoint tests, or browser tests of the frontend; the Smoke Test stage covers only that the deployed services start and are reachable. The gate is enforced by the order of the pipeline (Test before Build Images), so running docker compose build by hand builds only the production images and does not run the tests.

IX. ROLLBACK PROCEDURE AND DATA PERSISTENCE

Rollback

Every build produces images tagged with its build number (for example, iskolaria/proxy:12 and iskolaria/application-api:12), and the pipeline never prunes the tagged images. To return to an earlier build, the following commands are run from the folder that contains the real .env file, since the pipeline deletes its own copy for security reasons:

```
# list the available build tags
docker images "iskolaria/*"

# redeploy the images of build 9 without rebuilding
TAG=9 docker compose -p iskolaria up -d --no-build

# verify that the containers run the build 9 images
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
```

Figure 3

Verification of a Successful Build in Jenkins

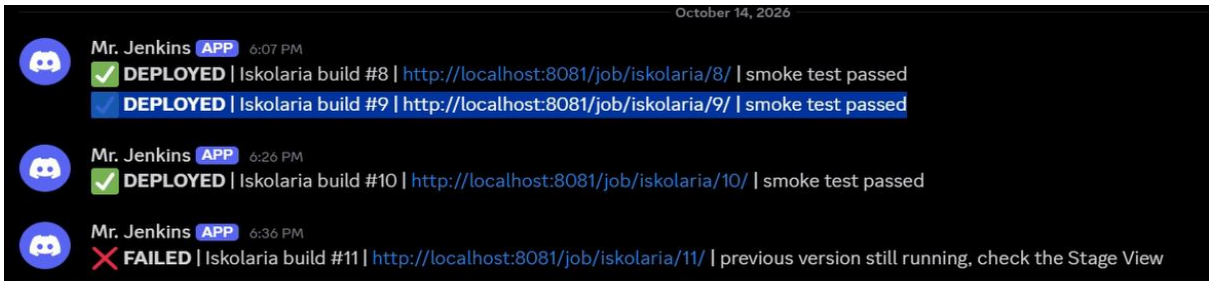


Figure 4

Verification of the Build Number in the Terminal

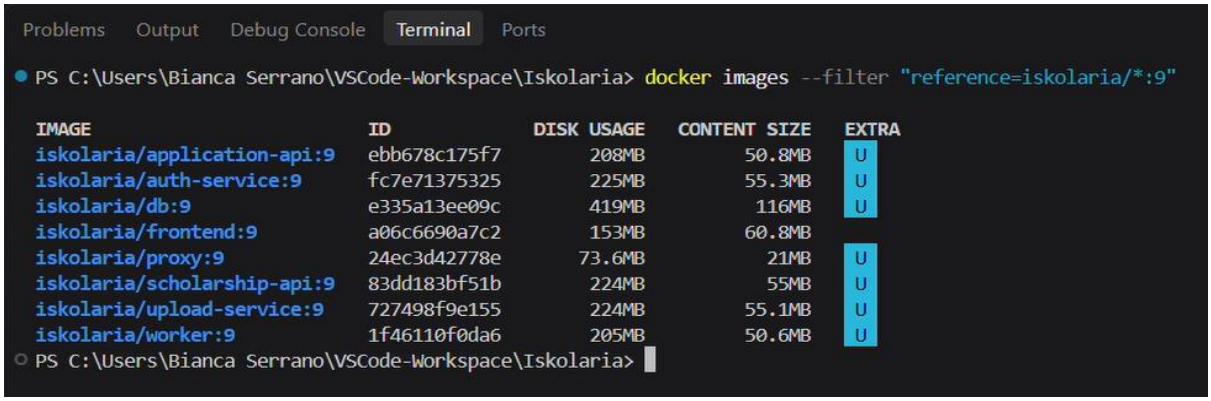


Figure 5

Redeployment of the Application Using Build Tag 9

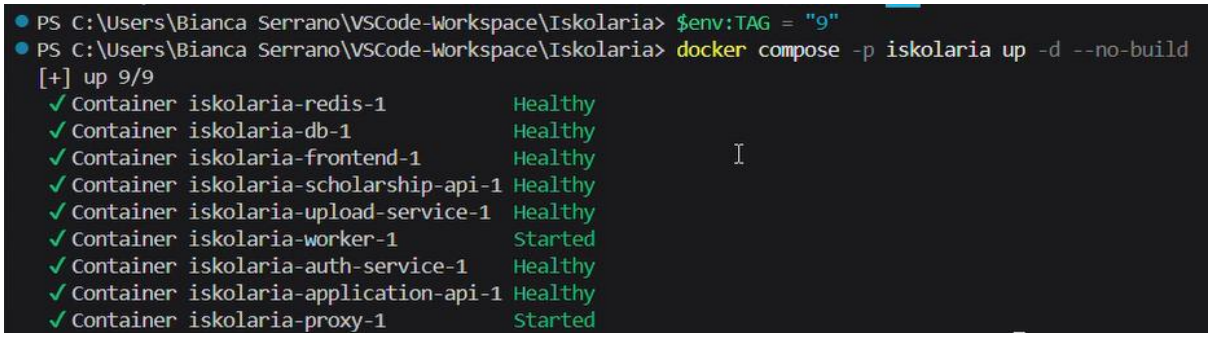


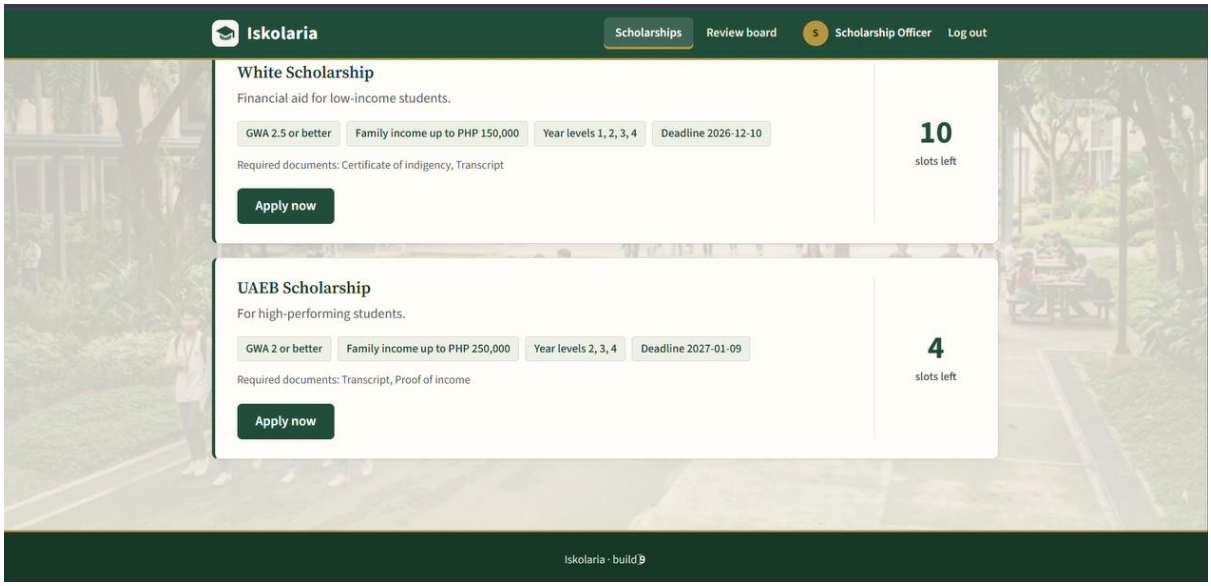
Figure 6

Verification That the Build Tag 9 Containers Are Running



Figure 7

Application Running on Build Tag 9



The footer of the website shows the build number, which proves the version that is currently running. The database volume is not touched during a rollback, so all data stays unchanged. A rollback restores the application code, not the data, so records created after the rolled-back build remain in the database. The rollback also stays in place only until the next push, because the following Jenkins build deploys the newest commit again.

Data Persistence

The state of the UAEB Scholarship was compared before and after a redeployment. In Build 10, one application was approved, which reduced the available slots of the UAEB Scholarship from five to four, and the resulting value was observed on both the applicant side and the administrator side. This change is stored in the PostgreSQL database, so it only survives a redeployment if the database files live outside the container. Afterward, a new build (Build 12) was triggered through the Jenkins pipeline, which replaced the containers of all built services, including the database container, without any manual export or restore. Once the Smoke Test passed, the same pages were checked again to determine whether the recorded data remained intact.

Figure 8

Build 10 shows that the UAEB Scholarship has 4 slots remaining.

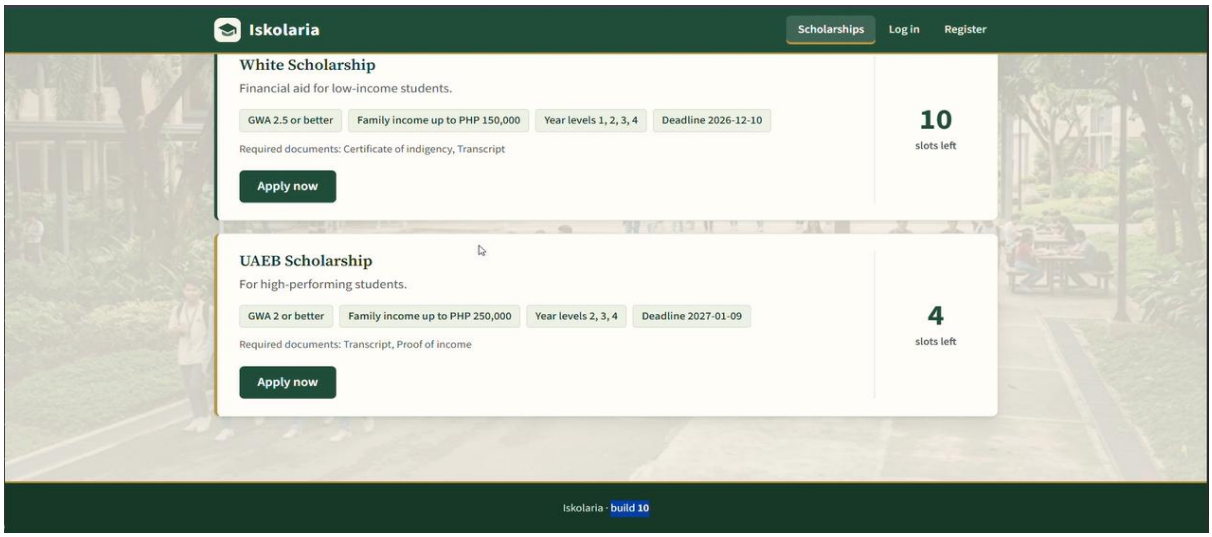


Figure 9

Admin view in Build 10 shows 4 of 5 slots remaining for the UAEB Scholarship.

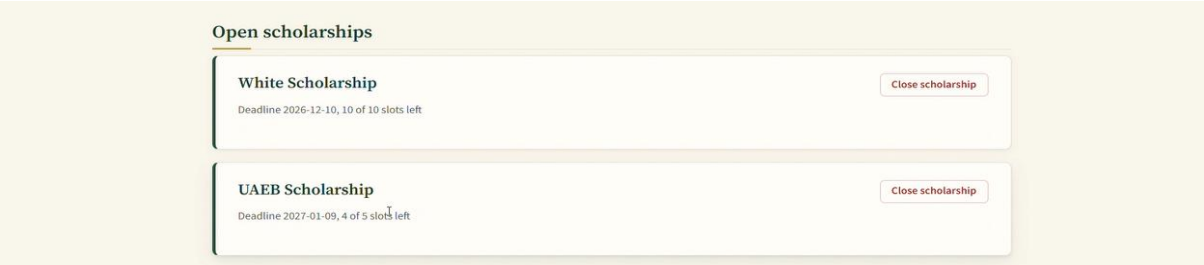


Figure 10

After building successfully in Build 12, the data persists and still shows 4 out of 5 slots.

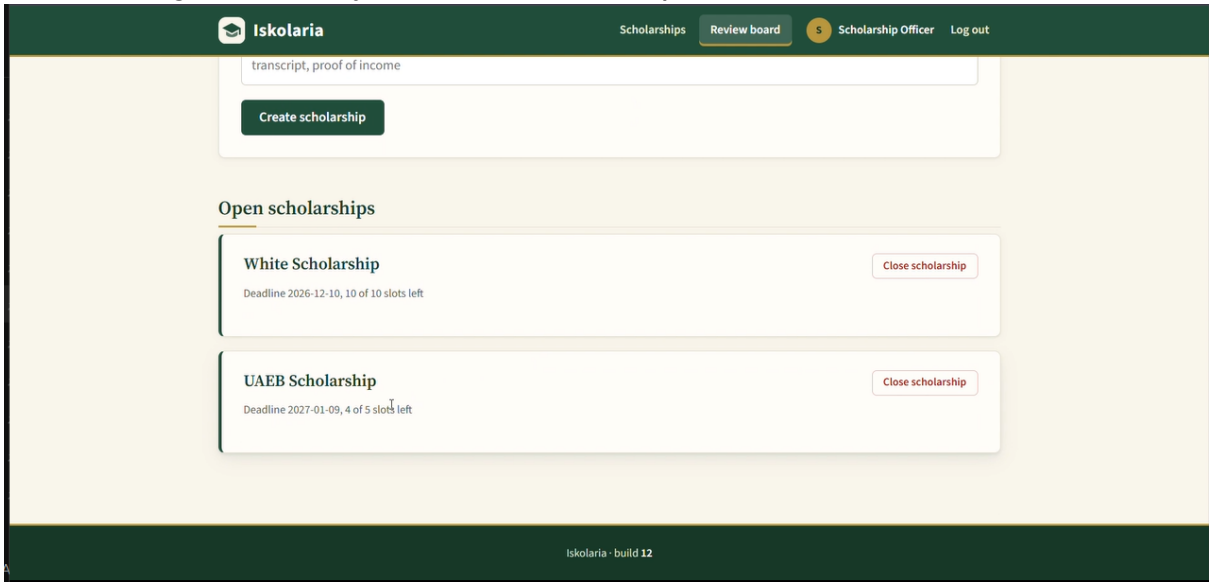
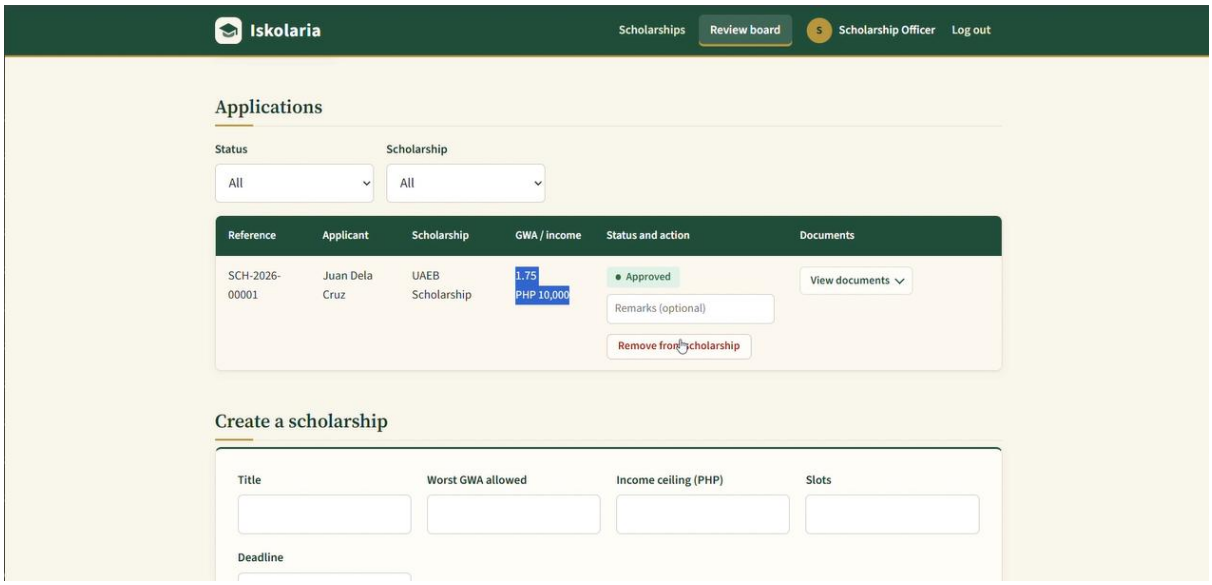


Figure 11

Data inside the administrator side persists after Build 12.



The results confirm that data persistence was achieved across deployments. Although the containers were rebuilt and replaced during Build 12, the slot count of 4 out of 5 remained unchanged on both the applicant and administrator sides. This behavior is made possible by the named Docker volume pg-data, which stores the PostgreSQL data outside the container's lifecycle.

X. PROBLEMS ENCOUNTERED

The following problems were found while building and testing the system, together with how each one was solved.

Problem	Solution
Jenkins triggers declared in the Jenkinsfile didn't fire	Triggers only register after the pipeline runs once, so the first build is started manually and later builds start from pushes.
The GitHub webhook cannot reach a local Jenkins	Added pollSCM('* * * * *') as a fallback that checks every minute.

Smoke test and health checks failed intermittently	busybox wget tried IPv6 (::1) first while nginx listens on IPv4 only, so the checks now use 127.0.0.1.
Removing an approved student lost the slot	Slots are returned on removal, capped with LEAST(slots_total, …).
Duplicate applications	UNIQUE (user_id, scholarship_id) plus a 409 response that returns the existing reference.
Students couldn't reapply after cancelling	ON CONFLICT DO UPDATE … WHERE status='cancelled' reuses the cancelled row.
Closed scholarships still accepted reviews and cancellations	is_open is checked before status changes and cancellations.
Impossible input (GWA 0, negative, above 5, NaN, negative income)	Range checks in check_eligibility, Pydantic Field(ge=1, le=5) and DB CHECK constraints, each with a test.

XI. INDIVIDUAL CONTRIBUTIONS

Iskolaria was developed by the four members of Team Ka Ba? as a group project for ITE302 (System Architecture and Integration). The system is made up of several independent modules, so the work was divided by area of specialization: infrastructure, CI/CD, project management, backend and database, and frontend, quality assurance, and documentation. This section presents how the work was divided and what each member delivered. Members' Roles and Responsibilities describes each member's role and main deliverables, and Section 11.2 summarizes their GitHub activity in the project repository. Together they show how the individual

parts were combined into one containerized system with an automated build and deployment pipeline.

Members' Roles and Responsibilities

MEMBER	ROLE	MAIN DELIVERABLES
Cabral, Bea Mei P.	Infrastructure Engineer	System architecture, Dockerfiles, docker-compose.yml, Nginx reverse proxy
David, Nigel James L.	DevOps / CI-CD Engineer	Jenkins in Docker, credentials, triggers, Jenkinsfile, Discord notifications
Serrano, Bianca Gabrielle R.	Project Lead / Scrum Master	Proposal, task board, Git repository management, demo flow
	Backend and Database Engineer	API modules, database schema and seed data, unit tests
Soriano, Den Mark C.	Frontend, QA, and Documentation Lead	Frontend pages, smoke test, README, technical documentation, slides

Individual Contributions and Deliverables

MEMBER	GITHUB ACCOUNT	COMMITS	PULL REQUESTS MERGED
Cabral, Bea Mei P.	beameipc49-blip	10	0
David, Nigel James L.	nijol-dvd	2	0
Serrano, Bianca Gabrielle R.	gxbiyel	45	9 (#1 and #9)
Soriano, Den Mark C.	dnmrkcs	12	0
TOTAL:		**69**	**9**

XII. REFERENCES

Docker, Inc. (n.d.). Compose file reference. Docker Docs. https://docs.docker.com/reference/compose-file/

Docker, Inc. (n.d.). Multi-stage builds. Docker Docs. https://docs.docker.com/build/building/multi-stage/

Jenkins. (n.d.). Installing Jenkins: Docker. Jenkins User Documentation. https://www.jenkins.io/doc/book/installing/docker/

Jenkins. (n.d.). Pipeline syntax. Jenkins User Documentation. https://www.jenkins.io/doc/book/pipeline/syntax/

Redis Ltd. (n.d.). BLMOVE. Redis Documentation. https://redis.io/docs/latest/commands/blmove/